

Refactoring Report Design Pattern Implementation

Course: IT342 – System Integration

Project: Disaster Aid Connect

Submitted by : Godwin M. Labaya

Date: 04/07/2026

Refactor 1: Singleton Pattern → supabaseClient.js (Formalized)

Before

The Supabase client was already being used as a singleton, but there was no centralized service layer. Every component called Supabase directly and inline, mixing data-fetching logic with UI rendering logic throughout Map.js, Requests.js, Dashboard.js, and Register.js.

Problems:

- If Supabase ever needed to be swapped or reconfigured, every file would need to be touched
- No single place to manage auth-related client behavior
- Business logic (e.g., "only fetch disasters for the current user") was duplicated across components

After — src/services/disasterService.js

```
// src/services/disasterService.js
// Singleton Pattern: one shared supabase instance is used across all service
calls.
// This centralizes all disaster-related data operations in one module.

import { supabase } from "../supabaseClient";

const disasterService = {
  async getAll() {
    const { data, error } = await supabase
      .from("disasters")
      .select("*")
      .order("created_at", { ascending: false });
    if (error) throw error;
    return data;
  },

  async getByUser(userId) {
    const { data, error } = await supabase
      .from("disasters")
```

```

        .select("*")
        .eq("created_by", userId)
        .order("created_at", { ascending: false });
    if (error) throw error;
    return data;
},

```

```

async create(payload) {
    const { data, error } = await supabase
        .from("disasters")
        .insert([payload])
        .select()
        .single();
    if (error) throw error;
    return data;
},

```

```

async update(id, updates) {
    const { data, error } = await supabase
        .from("disasters")
        .update(updates)
        .eq("id", id)
        .select()
        .single();
    if (error) throw error;
    return data;
},

```

```

async remove(id) {
    const { error } = await supabase
        .from("disasters")
        .delete()
        .eq("id", id);
}

```

```

        if (error) throw error;
    },
};

export default disasterService;

```

Usage in Map.js (After)

```

// Before (inline Supabase call scattered in Map.js):
// const { data, error } = await supabase
//   .from("disasters")
//   .select("*")
//   .order("created_at", { ascending: false });

// After (clean service call):
import disasterService from "../services/disasterService";

const fetchDisasters = useCallback(async () => {
  setLoading(true);
  try {
    const data = await disasterService.getAll();
    setDisasters(data);
    if (data.length > 0) setSelected((prev) => prev ?? data[0]);
  } catch (err) {
    console.error("Failed to fetch disasters:", err.message);
  } finally {
    setLoading(false);
  }
}, []);

```

Usage in Requests.js (After)

```

// Before:
// const { data, error } = await supabase
//   .from("disasters")

```

```

//   .select("*")
//   .eq("created_by", currentUID)
//   .order("created_at", { ascending: false });

// After:
import disasterService from "../../services/disasterService";

const fetchDisasters = useCallback(async () => {
  if (!currentUID) return;
  setLoading(true);
  try {
    const data = await disasterService.getByUser(currentUID);
    setDisasters(data);
  } catch (err) {
    console.error("Failed to fetch:", err.message);
  } finally {
    setLoading(false);
  }
}, [currentUID]);

```

Justification

The Singleton pattern ensures one shared Supabase instance handles all data operations. Formalizing the service layer separates concerns properly, components handle UI, the service handles data. If the backend ever changes (e.g., switching from Supabase to a custom REST API), only `disasterService.js` needs to be updated, not every individual component.

Improvements: Code organization, maintainability, reusability.

Refactor 2: Factory Pattern → `iconFactory.js`

Before

The `makeIcon()` function lived inside `Map.js`, embedded alongside hundreds of lines of component logic. It was not reusable by any other component. If a future page (e.g., a

legend or a list view) needed to render severity icons, it would have to duplicate this logic.

```
// Before — buried inside Map.js, not exportable/reusable
function makeIcon(color, isTemp = false) {
  const svg = `...`;
  return L.divIcon({ html: svg, className: "", iconSize: [32,44], ... });
}
```

Problems:

- Tightly coupled to Map.js; cannot be imported by other components
- If a new icon type is needed (e.g., a "verified" or "government-reported" marker), Map.js itself must be edited
- Color logic (getSeverityColor) was also duplicated between Map.js and Requests.js

After — src/utils/iconFactory.js

```
// src/utils/iconFactory.js
// Factory Pattern: centralizes icon creation logic.
// Callers pass a type or severity level; the factory returns the correct icon.

import L from "leaflet";

export function getSeverityColor(level) {
  if (level === "Critical") return "#7c3aed";
  if (level === "High")      return "#ef4444";
  if (level === "Medium")    return "#f97316";
  return "#3b82f6"; // Low
}

export function getSeverityStyle(level) {
```

```

    if (level === "Critical") return { bg: "#f3e8ff", text: "#7c3aed", border:
"#d8b4fe" };

    if (level === "High")      return { bg: "#fee2e2", text: "#dc2626", border:
"#fca5a5" };

    if (level === "Medium")    return { bg: "#ffedd5", text: "#ea580c", border:
"#fdb474" };

    return                      { bg: "#d1ecf1", text: "#20c997", border:
"#93c5fd" };
}

```

// Factory function – takes a severity level and optional flags, returns a Leaflet icon

```

export function createMarkerIcon(severityLevel, options = {}) {
  const { isTemp = false } = options;

  const color = getSeverityColor(severityLevel);

  const svg = `
    <svg xmlns="http://www.w3.org/2000/svg" width="32" height="44" viewBox="0
0 32 44">

      <filter id="s"><feDropShadow dx="0" dy="2" stdDeviation="2" flood-
opacity="0.25"/></filter>

      <path filter="url(#s)"

        d="M16 0C7.163 0 0 7.163 0 16c0 11.667 16 28 16 28S32 27.667 32 16C32
7.163 24.837 0 16 0z"

        fill="${color}" opacity="${isTemp ? 0.5 : 1}"/>

      <circle cx="16" cy="16" r="7" fill="white" opacity="0.9"/>

      ${isTemp
        ? `<line x1="16" y1="10" x2="16" y2="22" stroke="${color}" stroke-
width="2.5"/>`
        : ``}

      <line x1="10" y1="16" x2="22" y2="16" stroke="${color}" stroke-
width="2.5"/>`

    : ""}

    </svg>`;

  return L.divIcon({
    html: svg,

```

```

        className: "",
        iconSize: [32, 44],
        iconAnchor: [16, 44],
        popupAnchor: [0, -46],
    });
}

```

Usage in Map.js (After)

```

// Before:
// import { makeIcon, getSeverityColor } defined locally in Map.js

// After:
import { createMarkerIcon, getSeverityColor } from "../../utils/iconFactory";

// Creating a normal marker:
icon={createMarkerIcon(d.severity_level)}

// Creating a temporary pending pin:
icon={createMarkerIcon("Low", { isTemp: true })}

```

Usage in Requests.js (After)

```

// Before: getSeverityColor was copy-pasted and redefined in Requests.js

// After:
import { getSeverityStyle } from "../../utils/iconFactory";

const sev = getSeverityStyle(disaster.severity_level);

```


Justification

The Factory pattern extracts icon creation into a dedicated utility. Both Map.js and Requests.js previously redefined their own versions of severity color logic independently, this is a clear code smell. The factory eliminates duplication and makes it trivially easy to add new icon variants (e.g., a "government-verified" blue star marker) in one place.

Improvements: Reusability, maintainability, scalability.

Refactor 3: Observer Pattern → useAuth.js Custom Hook

Before

Authentication state retrieval was copy-pasted in multiple components. Both Dashboard.js and Requests.js (and implicitly Map.js) each ran their own `supabase.auth.getSession()` call on mount to figure out who was logged in. There was no shared, reactive auth state.

```
// Before - repeated in Dashboard.js:
```

```
useEffect(() => {  
  const getUser = async () => {  
    const { data: { session } } = await supabase.auth.getSession();  
    if (!session) { navigate("/"); return; }  
    // ... fetch username  
  };  
  getUser();  
}, [navigate]);
```

```
// And again in Requests.js:
```

```
useEffect(() => {  
  supabase.auth.getSession().then(({ data: { session } }) => {  
    if (!session) { navigate("/"); return; }  
    setCurrentUID(session.user.id);  
  });  
}, [navigate]);
```

```
// And again in Map.js:

useEffect(() => {
  supabase.auth.getSession().then(({ data: { session } }) => {
    setCurrentUID(session?.user?.id ?? null);
  });
}, []);
```

Problems:

- Auth logic is duplicated across at least three files
- No component automatically reacts to auth changes (e.g., session expiry, logout in another tab)
- Redirecting unauthenticated users is handled inconsistently

After — src/hooks/useAuth.js

```
// src/hooks/useAuth.js

// Observer Pattern: subscribes to Supabase's auth state change events.
// Any component using this hook automatically reacts when the auth state
changes,
// without polling or manually re-checking the session.

import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "../supabaseClient";

export function useAuth({ redirectIfUnauthenticated = true } = {}) {
  const [session, setSession] = useState(null);
  const [userId, setUserId] = useState(null);
  const [username, setUsername] = useState("");
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();
```

```

useEffect(() => {
  // Get the initial session
  supabase.auth.getSession().then(({ data: { session } }) => {
    if (!session && redirectIfUnauthenticated) {
      navigate("/");
      return;
    }
    setSession(session);
    setUserId(session?.user?.id ?? null);
    setLoading(false);
  });

  // Observer: subscribe to auth state changes
  // This fires whenever the user logs in, logs out, or the session refreshes
  const { data: { subscription } } = supabase.auth.onAuthStateChange(
    (_event, session) => {
      if (!session && redirectIfUnauthenticated) {
        navigate("/");
        return;
      }
      setSession(session);
      setUserId(session?.user?.id ?? null);
    }
  );

  // Cleanup: unsubscribe when the component unmounts
  return () => subscription.unsubscribe();
}, [navigate, redirectIfUnauthenticated]);

// Optionally fetch the username from the users table
useEffect(() => {

```

```

    if (!userId) return;

    supabase
      .from("users")
      .select("username")
      .eq("id", userId)
      .single()
      .then(({ data }) => { if (data) setUsername(data.username); });
  }, [userId]);

  return { session, userId, username, loading };
}

```

Usage in Dashboard.js (After)

```

// Before: manual session fetch + separate username fetch in useEffect

// After:
import { useAuth } from "../../hooks/useAuth";

export default function Dashboard() {
  const { username, loading } = useAuth({ redirectIfUnauthenticated: true });

  if (loading) return null;

  return (
    <div className="dashboard">
      {/* ... */}
      <h1>Welcome, {username || "Username"}!</h1>
      {/* ... */}
    </div>
  );
}

```

```
}
```

Usage in Requests.js (After)

```
// Before: supabase.auth.getSession() manually called in useEffect
```

```
// After:
```

```
import { useAuth } from "../../hooks/useAuth";
```

```
export default function RequestsPage() {
```

```
  const { userId: currentUID, loading: authLoading } = useAuth();
```

```
  // currentUID is now reactive - if the session expires, this updates automatically
```

```
  // ...
```

```
}
```

Usage in Map.js (After)

```
// Before: supabase.auth.getSession() in its own useEffect
```

```
// After:
```

```
import { useAuth } from "../../hooks/useAuth";
```

```
export default function MapPage() {
```

```
  const { userId: currentUID } = useAuth({ redirectIfUnauthenticated: false });
```

```
  // Map is still viewable by guests; auth is optional here
```

```
  // ...
```

```
}
```

Justification

The Observer pattern, implemented via Supabase's `onAuthStateChange` subscription inside a custom hook, means every component automatically reacts to auth events rather than blindly reading a session snapshot once on mount. The custom hook also eliminates the most egregious duplication in the project, the same 8-line session-fetching block appeared in at least three separate files. Now it lives in one place, is testable in isolation, and handles edge cases like session expiry consistently.

Improvements: Code organization, maintainability, reusability, scalability.

Summary Table

Pattern	Where applied	File(s) Changed
Singleton	Supabase service layer	services/disasterService.js, Map.js, Requests.js
Factory	Icon and color creation	utils/iconFactory.js, Map.js, Requests.js
Observer	Auth state subscription	hooks/useAuth.js, Dashboard.js, Map.js, Requests.js